

Architectural and Empirical Review of the Enterprise Agentic Streaming Lakehouse

The proposed engineering capstone, the "Enterprise Agentic Streaming Lakehouse," represents a highly sophisticated convergence of event-driven distributed systems, modern open table formats, and agentic artificial intelligence. Executing this locally in Jheepasani, Rajasthan, utilizing constrained Docker hardware poses a formidable engineering challenge. Deploying a continuous integration pipeline that captures transactional database mutations, processes them with exactly-once semantics, and exposes the aggregated results to a Large Language Model (LLM) via the Model Context Protocol (MCP) aligns perfectly with the bleeding edge of the 2026 data engineering landscape.

However, evaluating this architecture through the strict constraints of a ten-week academic timeline, a four-engineer undergraduate team, and the severe physical resource limitations of local computational environments reveals critical operational risks. The initial design document contains theoretically sound paradigms but suffers from over-engineering in certain orchestration layers while under-specifying mandatory components like catalog concurrency and metadata maintenance. This comprehensive technical analysis deconstructs the proposed architecture, challenges its foundational assumptions utilizing current empirical research, and establishes a rigorous experimental framework. The objective is to elevate this capstone from a standard systems implementation into a measurable, academically rigorous experiment evaluating deterministic semantic layers against generative AI in streaming environments.

1. Deep Technical Analysis of the Current Architecture

The data lifecycle from operational genesis to analytical inference is continuous and unbounded. Evaluating each component requires a deep analysis of its internal mechanics, its interfaces, its failure modes, and its theoretical justification within a highly constrained local Docker environment.

The Operational and Ingestion Plane

The pipeline initiates at the transactional system of record and utilizes Change Data Capture (CDC) to stream mutations as discrete events.

PostgreSQL (System of Record): PostgreSQL serves as the operational database simulating e-commerce transactions. Its inclusion is foundational because it supports Write-Ahead Logging (WAL) and logical decoding via the pgoutput plugin¹. This architecture allows downstream systems to read a sequential, durable log of physical row changes rather than relying on highly inefficient timestamp-based batch polling. However, a critical failure mode in local deployments involves the `max_slot_wal_keep_size` configuration. If the downstream CDC engine disconnects or stalls, PostgreSQL will retain WAL files indefinitely to satisfy the logical replication slot, rapidly causing catastrophic out-of-disk-space failures on local development

machines³. Configuring this parameter forces the database to aggressively sever the replication slot if the WAL backlog threatens system stability, sacrificing downstream continuity to preserve the primary operational database.

Debezium Server (CDC Generation): Debezium represents the undisputed industry standard for capturing database events. The original architecture appropriately specified Debezium, but standard enterprise deployments typically embed Debezium within a distributed Apache Kafka Connect cluster. Deploying Kafka Connect locally requires a dedicated Java Virtual Machine (JVM), specific internal distributed topics, and a massive memory footprint⁵. The shift to Debezium Server running as a standalone Quarkus application is highly recommended and practically mandatory for this capstone. Quarkus enables a micro-footprint, ahead-of-time compiled deployment that often consumes less than 150MB of RAM⁷. This standalone service captures INSERT, UPDATE, and DELETE events directly from the PostgreSQL WAL and serializes them into JSON payloads without the crushing overhead of a full Connect cluster⁹.

Redpanda (Event Broker): The selection of Redpanda over Apache Kafka is a profound optimization for constrained local hardware. Redpanda is written entirely in C++ and implements the Raft consensus algorithm directly, entirely bypassing the JVM, garbage collection pauses, and the need for external metadata controllers like ZooKeeper or KRaft¹¹. It serves as the durable, high-throughput buffer between the operational and analytical planes. If the downstream stream processor crashes, Redpanda ensures that CDC events are durably retained on disk for exact replay. Redpanda also provides the essential routing infrastructure for the Dead Letter Queue (DLQ), buffering malformed records for later inspection without halting the primary ingestion pipeline¹.

The Streaming and Lakehouse Plane

This segment manages data quality enforcement, stateful temporal transformations, and transactional metadata commits into the analytical storage layer.

Apache Flink (Stream Processor): Apache Flink is strictly required for this architecture due to its unparalleled ability to maintain stateful processing and execute exactly-once delivery semantics over unbounded streams¹². Flink consumes the continuous Redpanda topics, applies declarative SQL validation filters to detect malformed data, and dynamically routes invalid records to the DLQ. Flink achieves exactly-once semantics into the lakehouse via a Two-Phase Commit (2PC) protocol perfectly aligned with its distributed checkpointing mechanism¹⁵. Flink buffers CDC records in its managed state, writes physical Parquet data files to object storage continuously, but only commits the logical metadata transaction to the catalog upon the successful completion of a global checkpoint barrier¹⁶.

MinIO (Object Storage) and Apache Iceberg (Table Format): MinIO provides high-performance, S3-compatible local object storage, serving as the raw storage substrate for the lakehouse. Apache Iceberg overlays this raw storage with an open table format, providing ACID (Atomicity, Consistency, Isolation, Durability) transactions, hidden partitioning, and schema evolution⁶. Because the pipeline processes continuous streaming updates from Debezium, the Iceberg Flink sink must be configured for "upsert" mode¹⁶. By mapping PostgreSQL primary keys to Iceberg equality fields, Iceberg utilizes Merge-on-Read (MoR)

semantics. It writes new rows as standard data files and updates/deletes as separate "equality delete" files, entirely avoiding the massive latency penalty of Copy-on-Write (CoW) full-file rewrites during high-frequency streaming ingestion¹⁸.

Apache Polaris (REST Catalog): The initial architecture listed "Iceberg/MinIO" without explicitly defining a catalog mechanism, which is a catastrophic conceptual gap. An Iceberg table is fundamentally a hierarchical tree of files (metadata.json, manifest lists, manifests, and Parquet data). Without a centralized concurrency controller, distributed writers (Flink) and readers (Trino) will experience immediate race conditions and metadata corruption¹. Apache Polaris implements the open Iceberg REST Catalog specification, providing a unified HTTP endpoint⁶. Polaris acts as the ultimate arbiter of truth, utilizing a backend relational database to manage optimistic concurrency control and ensuring Flink's commits never conflict with Trino's read operations.

Critical Missing Component - Post-Commit Table Maintenance: Continuous streaming with Merge-on-Read semantics causes extreme physical file fragmentation. Every single Flink checkpoint generates microscopic data files and equality delete files¹⁵. Query engines reading this highly fragmented state must perform computationally expensive on-the-fly merges, which quickly destroys read latency and exhausts memory²¹. The original architecture implies the use of external orchestrators for maintenance. However, the Apache Iceberg Flink V2 Sink implementation features an `addPostCommitTopology` hook, allowing the Flink cluster itself to execute background `RewriteDataFiles` (compaction) and `ExpireSnapshots` operations automatically in the post-commit phase¹³. Implementing this native hook is vital, rendering external batch orchestrators completely unnecessary and preserving critical Docker compute resources.

The Semantic and Agentic Plane

This layer securely exposes the highly optimized analytical data to the AI agent.

Trino (MPP Query Engine): Trino serves as the Massively Parallel Processing query engine, perfectly embodying the separation of compute from storage²⁴. Trino connects to Polaris via the REST catalog connector to resolve Iceberg metadata and executes highly parallelized, vectorized ANSI SQL queries directly over the Parquet files in MinIO⁶.

FastMCP (Model Context Protocol Server): FastMCP acts as the deterministic API gateway connecting the LLM to the database. Utilizing standard I/O (stdio) or Server-Sent Events (SSE) transports²⁵, FastMCP allows developers to define strict Python functions decorated with Pydantic schemas²⁷. This forces the LLM to adhere to a rigid JSON-RPC communication contract. The server exposes a schema-fetching tool to inject context into the LLM prompt (Retrieval-Augmented Generation), preventing the model from hallucinating table names¹.

Sqlglot (Deterministic Security Validation): Passing unconstrained, AI-generated SQL directly to a distributed query engine presents an unacceptable enterprise security vulnerability¹. The architecture correctly implements sqlglot, a pure-Python SQL parser and transpiler. Before any LLM string touches the Trino driver, the FastMCP server uses sqlglot to parse the string into an Abstract Syntax Tree (AST). The Python server then computationally traverses this AST using a visitor pattern to ensure the root node is strictly an `exp.Select`

expression, actively blocking any `exp.Drop`, `exp.Delete`, `exp.Update`, or `exp.Alter` nodes¹. This deterministic guardrail proves a deep understanding of AI safety engineering.

2. Engineering Workload and Dependency Matrix

To ensure the four-person undergraduate team delivers a highly fault-tolerant system within ten weeks, technical responsibilities must be strictly decoupled to prevent integration bottlenecks while maintaining shared accountability for the end-to-end flow.

Engineer 1: The Operational Architect (PostgreSQL, Data Generation, CDC)

Engineer 1 is responsible for the genesis of all system data. They must deploy PostgreSQL, configure WAL retention policies, and implement the dynamic data mutation pipeline. Crucially, they must configure the standalone Debezium Server via Quarkus application properties to capture WAL events efficiently without exhausting local memory⁷.

- **Concepts to Master:** PostgreSQL logical decoding, replication slots, CDC JSON serialization formats, Docker networking, and relational schema normalization.
- **Inputs / Outputs:** Ingests raw Python data scripts; outputs continuous, well-formed CDC event streams into isolated Redpanda topics.
- **Testing & Reliability:** Must physically demonstrate that Debezium resumes gracefully after a forced Docker container restart without dropping or missing any database transactions.
- **Definition of Done (DoD):** Continuous, real-time CDC events are visible in the Redpanda Console mirroring the exact state of the PostgreSQL schema, operating stably under strict Docker memory limits.
- **Major Risks:** WAL accumulation crashing the Docker host if Debezium fails.

Engineer 2: The Stream Processing Engineer (Redpanda, Flink, Data Quality)

Engineer 2 owns the streaming transformation and quality enforcement layer. They must deploy Redpanda and the Apache Flink MiniCluster. They are tasked with writing declarative Flink SQL to consume Redpanda topics, enforcing data quality rules (e.g., filtering negative transactions or null identifiers), and sinking data into Iceberg via the REST catalog¹².

- **Concepts to Master:** Exactly-once processing, event-time watermarking, Two-Phase Commit protocols, Dead Letter Queue (DLQ) routing, and Iceberg equality fields for UPSERT streaming.
- **Inputs / Outputs:** Consumes raw JSON streams from Redpanda; outputs structurally valid Parquet records to MinIO and routes malformed validation failures to a separate Redpanda DLQ topic.
- **Testing & Reliability:** Must demonstrate that intentionally injected malformed records from Engineer 1 are successfully caught by Flink SQL WHERE clauses and routed to the DLQ, ensuring only pristine data enters the lakehouse¹³.
- **Definition of Done (DoD):** Data continuously populates the Iceberg table with

exactly-once guarantees, surviving simulated Flink task manager failures via distributed checkpoints.

- **Major Risks:** State bloat in Flink causing Out-of-Memory (OOM) errors during checkpointing.

Engineer 3: The Lakehouse and Semantic Architect (MinIO, Iceberg, Polaris, Trino)

Engineer 3 is the custodian of the analytical storage and semantic layer. They must deploy MinIO, instantiate Apache Polaris, and configure Trino. A critical responsibility is configuring the Flink Iceberg V2 Sink to utilize `addPostCommitTopology` for automated small-file compaction, preventing the lakehouse from collapsing under its own metadata weight¹³.

- **Concepts to Master:** Object storage primitives, the Iceberg REST Catalog specification, Merge-on-Read (MoR) equality delete mechanics, metadata hierarchies (manifest lists vs. data files), and Trino query optimization¹.
- **Inputs / Outputs:** Takes raw Parquet byte streams and metadata commits from Flink; provides a highly performant, fully ACID-compliant ANSI SQL interface via Trino.
- **Testing & Reliability:** Must prove query latency remains consistently stable over time by demonstrating successful, automated background compaction of Iceberg equality delete files²².
- **Definition of Done (DoD):** Trino successfully executes complex aggregations over the Iceberg tables in sub-second latency, and the MinIO storage bucket does not exhibit uncontrolled small-file proliferation.
- **Major Risks:** Catalog lock conflicts between Flink writers and background compaction threads.

Engineer 4: The AI Systems & Evaluation Engineer (FastMCP, Sqlglot, Benchmarks)

Engineer 4 bridges the gap between the pristine data platform and the end-user. They must develop the FastMCP Python server, exposing the Trino database securely. They will implement the schema-fetching tool for Retrieval-Augmented Generation (RAG) and utilize sqlglot to computationally block destructive SQL. Furthermore, they own the execution of the final LLM vs. Non-LLM benchmark experiment¹.

- **Concepts to Master:** Model Context Protocol (JSON-RPC), Abstract Syntax Trees (AST), prompt engineering, text-to-SQL evaluation metrics, and experimental design²⁵.
- **Inputs / Outputs:** Takes natural language questions from the user; outputs deterministic, safe SQL to Trino, and returns tabulated analytical results to the LLM context window.
- **Testing & Reliability:** Must successfully intercept and block simulated SQL injection attacks using programmatic AST traversal before the query string is dispatched to the Trino JDBC driver¹³.
- **Definition of Done (DoD):** The LLM autonomously answers complex analytical queries by discovering the schema, generating verifiable SQL, and successfully executing the formal 50-question benchmark.

- **Major Risks:** Unpredictable LLM hallucinations causing catastrophic test failures during the final demonstration.

Ownership and Dependency Matrix

Phase / Component	Primary Owner	Downstream Dependency	Integration Requirement
Operational Schema Design	Engineer 1	Engineer 4	The schema dictates the complexity of the LLM's Text-to-SQL reasoning paths.
CDC Event Serialization	Engineer 1	Engineer 2	Debezium JSON payloads must perfectly match the Flink SQL DDL definitions.
Data Quality & Anomaly Injection	Engineer 1 & 2	Engineer 3	Flink must successfully filter anomalies before they corrupt the Iceberg catalog.
Iceberg Sink & Compaction	Engineer 3	Engineer 2	addPostCommitTo topology must be seamlessly integrated into Engineer 2's Flink job.
Trino & Polaris Connectivity	Engineer 3	Engineer 4	Trino must provide stable JDBC/REST connections for FastMCP to execute queries.
LLM Execution & AST Safety	Engineer 4	All Engineers	The AI layer relies on the latency, data quality, and security

			of the entire underlying stack.
--	--	--	---------------------------------

3. Dataset Strategy: Transcending Synthetic Limitations

The original architectural proposal implied the use of a Python Faker script to generate entirely synthetic data. From an academic and empirical standpoint, this approach is critically flawed. Purely synthetic generators inevitably produce uniform statistical distributions, perfect referential integrity, and simplistic relational structures. These sterile conditions fail to rigorously test distributed stream processors, data quality DLQ routing, or the advanced multi-hop reasoning capabilities of Text-to-SQL models¹⁹. Furthermore, evaluating LLMs on synthetically generated, clean data drastically overstates their real-world capabilities.

Benchmark Dataset Analysis

To understand the requirements for a proper dataset, we must look at how the industry evaluates Text-to-SQL models:

- **WikiSQL:** An older benchmark consisting of highly simplistic, single-table queries. It is effectively obsolete for modern enterprise evaluation¹⁹.
- **Spider 2.0:** Released in 2024, this benchmark represents enterprise workflows involving multi-dialect SQL over cloud databases that frequently exceed 1,000 columns. It emphasizes extreme complexity and long-context reasoning³³.
- **BIRD (Big Bench for Large-scale Database Grounded Text-to-SQL Evaluation):** BIRD represents the gold standard for evaluating dirty, real-world data. It includes messy string values, missing documentation, and external business logic. Crucially, human data engineers score 92.96% Execution Accuracy on BIRD, while the best LLMs hover around 81%, illustrating a massive gap in AI reasoning¹⁹.
- **BIRD-Interact:** A 2026 extension of BIRD that tests conversational protocols, where LLMs must autonomously query a user simulator to resolve ambiguous schema definitions before writing SQL³⁶.

Recommended Strategy: The Time-Dilated Olist Hybrid Approach

Implementing Spider 2.0 or BIRD directly within a 10-week local Docker capstone is computationally unfeasible. The optimal strategy is a hybrid methodology utilizing the **Olist Brazilian E-Commerce Public Dataset**³⁸.

The Olist dataset contains approximately 100,000 real, anonymized orders spanning multiple years across eight highly relational tables (Orders, Order Items, Payments, Reviews, Products, Sellers, Customers, Geolocation). At approximately 120MB in raw CSV format, it is large enough to demonstrate meaningful analytical aggregations but small enough to prevent local Out-of-Memory failures⁴⁰.

Why Olist is Academically Superior:

- 1. **Relational Complexity:** The schema demands complex multi-hop joins (e.g., joining Customers to Orders to Items to Products to English Translations to calculate revenue by category)⁴². This provides a rigorous, BIRD-like test for the AI agent.
- 2. **Genuine Data Decay:** Olist contains real-world anomalies that cannot be easily faked—null delivery timestamps for canceled orders, Portuguese text requiring translation, and delayed fulfillment³⁸. This perfectly justifies Engineer 2's Flink DLQ architecture.

The Hybrid Execution Mechanism: Instead of loading the Olist CSV files statically into PostgreSQL, Engineer 1 will construct a Python streaming mutator. This script will load the static data into memory, sort the records chronologically by order_purchase_timestamp, and execute INSERT, UPDATE, and DELETE statements against the PostgreSQL database in a time-dilated continuous loop⁴². For example, an order will first be INSERTed with a status of 'processing', and exactly 30 seconds later, an UPDATE will change the status to 'shipped'. This approach accurately simulates a high-velocity operational database, fully exercising Debezium's log-tailing capabilities, Flink's exactly-once state management, and Iceberg's complex Merge-on-Read equality delete mechanics.

Dataset Attribute	Python Faker (Original)	Static Benchmark (BIRD)	Time-Dilated Olist (Recommended)
Relational Depth	Extremely Low (Flat)	Extremely High (1000+ cols)	High (8-table Snowflake schema)
Data Quality Realism	None (Perfect data)	High (Dirty string values)	High (Nulls, missing timestamps)
CDC Simulation	Synthetically easy	None (Static evaluation)	Perfect (Chronological replay)
Hardware Feasibility	High	Low (Requires cloud scaling)	High (120MB raw data)

4. Developing the Experimental Baselines: LLM vs. Non-LLM

A standard engineering capstone builds a pipeline; an exceptional capstone builds a pipeline and utilizes it to answer a novel research question. The professor's suggestion to compare LLM vs. Non-LLM approaches elevates the project into an empirical study. Recent studies on Spider

2.0 and BIRD demonstrate that LLM execution accuracy drops precipitously when confronted with real-world schemas, often suffering from severe hallucination and schema linking failures³³.

To turn this into a genuine experimental contribution, the team will implement and benchmark three distinct analytical retrieval baselines over the identical Trino/Iceberg backend.

Baseline 1: Frontier LLM with RAG (The Architecture Default)

This baseline relies on generative AI. The FastMCP server dynamically retrieves the database schema (tables, columns, data types) from Trino's `information_schema`, injects it into the prompt via Retrieval-Augmented Generation (RAG), and relies entirely on the LLM's reasoning to generate the SQL query¹³.

- **Model:** Claude 3.5 Sonnet or OpenAI GPT-4o.
- **Hypothesis:** Offers maximum linguistic flexibility and zero-shot reasoning, but will suffer from high token costs, high latency, and hallucinated table joins on complex Olist queries.

Baseline 2: Open-Weight Small Language Model (SLM)

Enterprise organizations frequently require on-premises, open-weight models to maintain strict data privacy compliance⁴⁶. This baseline evaluates a highly specialized local coding model.

- **Model:** Qwen2.5-Coder-7B-Instruct⁴⁷. Research indicates this 7-billion parameter model is exceptionally capable, occasionally surpassing 84% accuracy on select code evaluations and demonstrating highly competitive performance on BIRD when utilizing Chain-of-Thought reasoning pipelines⁴⁹.
- **Hypothesis:** Reduces API costs to zero and maintains strict data residency, but may struggle with deep semantic multi-hop reasoning compared to massive frontier cloud models.

Baseline 3: Deterministic Semantic Layer (Non-LLM Baseline)

To avoid the Text-to-SQL accuracy cliff, modern data engineering is increasingly shifting toward Semantic Layers. A semantic layer defines entities, measures, and complex join paths in a compiled, deterministic domain-specific language (DSL)⁵¹.

- **Technology:** Malloy (an open-source language developed by Google)⁵⁴.
- **Implementation:** Engineer 4 will write a static, programmatic Malloy model defining the exact relationships between the 8 Olist tables. The user's query will be routed to a pre-defined Malloy function, which the Malloy compiler then deterministically translates into flawless, highly optimized Trino SQL⁵⁶. This is not a "strawman" baseline; it represents the current enterprise standard for guaranteed accuracy.
- **Hypothesis:** Yields 100% execution accuracy and absolute zero hallucinations for supported questions, ensuring compute-correct generation⁵⁸. However, it completely lacks the flexibility to answer novel, ad-hoc questions outside the bounds of the pre-compiled semantic model.

(Note: Advanced reinforcement learning pipelines like ReViSQL, which utilize RLVR and inference-time scaling via majority voting to achieve 93.2% EX on BIRD⁵⁹, represent the

bleeding edge of AI research. While too complex for a 10-week undergraduate build, acknowledging these techniques in the final thesis demonstrates profound domain awareness).

5. Designing a Rigorous Experimental Framework

To guarantee academic rigor, the comparison between the three baselines must be scientifically controlled, utilizing the identical Trino compute engine, the identical Iceberg data state, and a highly structured benchmarking metric suite.

The Custom Olist Benchmark Suite

Recent 2026 research utilizing AI evaluation agents (such as SAR-Agent) indicates that major public datasets like BIRD suffer from massive annotation error rates, ranging from 32% to 61%¹¹. These pervasive annotation errors significantly distort leaderboard rankings and misguide research⁶¹. Therefore, relying strictly on an automated subset of public benchmarks is risky. Instead, Engineer 4 will author a custom, expert-verified benchmark suite of 50 natural language questions specifically tailored to the Olist schema. The questions will be strictly categorized by difficulty:

1. **Simple (15 questions):** Single-table aggregations and basic filtering (e.g., "What was the total revenue generated by Olist in 2017?").
2. **Moderate (15 questions):** Two-table joins with dimensional grouping (e.g., "List the top 5 product categories by total sales volume in São Paulo.").
3. **Complex (10 questions):** Multi-hop joins involving subqueries, window functions, or complex temporal logic (e.g., "What is the average delivery delay for orders over \$100 paid by credit card?").
4. **Adversarial (10 questions):** Ambiguous requests, queries utilizing non-standard Portuguese terminology, and explicit SQL injection attempts to test the AST parser¹³.

Evaluation Metrics

The evaluation will adapt metrics from the BIRD and Spider 2.0 research frameworks to measure system efficacy quantitatively¹⁹:

Metric	Definition	Engineering Justification
Execution Accuracy (EX)	Measures whether the result set produced by the generated SQL exactly matches the gold-standard result set, ignoring syntactic query differences ⁵ .	Standard string-matching fails because multiple distinct SQL queries can produce the exact same accurate result. EX is the definitive ground-truth metric.
Fractional Execution	A softer metric measuring	Provides partial credit for

Accuracy	the proportion of the result columns matching the golden result ²⁹ .	queries that successfully locate the right tables but miss specific aggregation requirements.
System Latency	Measures the end-to-end time from query submission to result rendering in the chat interface.	Captures the compounding delays of FastMCP processing, LLM Token generation, and Trino Iceberg execution.
Security Violation Rate	Measures the percentage of times the LLM attempts to execute a destructive mutation (e.g., DROP), and confirms whether sqlglot successfully blocked it ¹ .	Proves the efficacy of the deterministic safety layer over the non-deterministic generative layer.

To ensure experimental fairness, the underlying streaming data pipeline must be paused during benchmark execution. Evaluating models against an actively mutating Iceberg table introduces non-deterministic data states, making Execution Accuracy (EX) comparisons scientifically invalid.

6. Investigation of the Hybrid Tiered Architecture

The initial project proposal inquired whether implementing a heavily tiered pipeline—specifically, a "Semantic cache -> deterministic/non-LLM system -> lightweight model -> LLM fallback" architecture—provides meaningful research value or simply introduces unnecessary complexity.

Assessment: Remove from Scope due to Systemic Complexity. While utilizing a vector database for semantic caching (performing a similarity search to return previously validated SQL for repetitive natural language questions) reduces LLM compute costs and latency in static environments, it introduces severe, often fatal architectural complexity in a streaming lakehouse¹.

In this architecture, the underlying Apache Iceberg table is continuously mutated by Debezium CDC events, with new snapshots committed every minute via Flink checkpoints. A semantic cache requires incredibly complex invalidation logic whenever the underlying operational schema evolves or when data updates render previously cached answers logically stale. Furthermore, cascading a query through four separate fallback layers introduces multiple points of network latency, complex error handling, and orchestration nightmares. The experimental value is far higher when comparing the three baselines (Frontier LLM vs. SLM vs. Semantic Layer) as distinct, independent FastMCP tools. The FastMCP server should expose all three engines parallelly, allowing the user to dictate which engine handles the query. This

ensures clear, unambiguous attribution of errors and latency during the benchmarking phase.

7. Reworking the Project Scope for Feasibility

In a highly constrained ten-week academic project restricted by local Docker hardware, scope creep is the primary cause of catastrophic system failure. The team must be ruthless in prioritizing the core streaming pipeline over peripheral features.

Scope Rationalization Matrix

Feature / Component	Decision	Architectural Justification
Apache Flink DLQ Routing	Keep	Essential. Demonstrates advanced shift-left data quality engineering by filtering Olist anomalies ¹³ .
FastMCP & sqlglot AST Parsing	Keep	Mandatory. Passing unconstrained LLM SQL to Trino is insecure. AST parsing ensures deterministic safety ¹³ .
Apache Polaris REST Catalog	Keep	Mandatory. Iceberg requires a catalog for atomic commits and concurrent read/write locks between Flink and Trino ¹ .
Apache Flink Iceberg Maintenance	Simplify	Use Flink V2 Sink addPostCommitTopology for compaction. Do not attempt to build standalone Spark/Airflow maintenance jobs ¹³ .
External Orchestrators (dbt/Airflow)	Remove	Over-engineered. This is a streaming architecture; batch orchestration directly contradicts the real-time paradigm ⁶⁴ .
Custom Web Frontend	Remove	Massive distraction from

(React/Vue)		core data engineering. Rely on Trino CLI, Redpanda Console, and standard MCP clients (Claude Desktop).
Semantic Caching Layer	Remove	State management and cache invalidation are too complex for a continuously mutating streaming database.

Revised 10-Week Engineering Roadmap

To ensure academic safety, the roadmap is explicitly structured to achieve a Minimum Viable Product (MVP) by Week 5. Securing the MVP ensures the team possesses a functional system to demonstrate, even if the advanced AI experiments encounter delays.

- **Week 1: Foundation.** Engineer 1 builds the docker-compose.yml for PostgreSQL, MinIO, and Redpanda. Initiates the Olist dataset Python mutator script.
- **Week 2: CDC Ingestion.** Engineer 1 deploys standalone Debezium Server. Configures PostgreSQL logical replication slots. Verifies continuous Olist CDC payloads in Redpanda.
- **Week 3: Lakehouse Storage.** Engineer 3 deploys Apache Polaris and Trino. Configures MinIO buckets for Iceberg Parquet data. Establishes the REST catalog connection.
- **Week 4: Streaming MVP.** Engineer 2 deploys Flink MiniCluster. Writes base Flink SQL to consume Redpanda topics and execute exactly-once writes into Iceberg.
- **Week 5: System Integration (MVP ACHIEVED).** The team proves that simulated Olist mutations in PostgreSQL successfully propagate through Flink and are queryable via Trino.
- **Week 6: Data Quality & Maintenance.** Engineer 2 implements Flink DLQ routing for malformed Olist records. Engineer 3 configures Flink addPostCommitTopology to automate Iceberg compaction.
- **Week 7: AI Infrastructure.** Engineer 4 develops the FastMCP Python server. Implements the get_schema_context RAG tool to pull Trino metadata into the LLM context.
- **Week 8: AI Safety Layer.** Engineer 4 integrates sqlglot. Implements recursive AST parsing to actively block exp.Drop and exp.Update commands, returning errors to the LLM.
- **Week 9: Benchmarking & Experimentation.** The team executes the 50-question Olist benchmark across the Frontier LLM, Qwen2.5 SLM, and Malloy Semantic Layer baselines. Records Execution Accuracy and latency metrics.
- **Week 10: Synthesis & Documentation.** Finalize architecture diagrams, write the thesis analyzing the benchmark results, and record the final demonstration video.

8. Final Project Recommendations and Conclusion

The proposed "Enterprise Agentic Streaming Lakehouse" transcends a typical undergraduate capstone. By capturing continuous operational mutations, processing them with distributed exactly-once semantics, and subjecting the resulting analytical layer to a rigorous AI capability benchmark, the project addresses the precise friction points of the 2026 enterprise data stack. To maximize data engineering depth, academic rigor, and demonstrable portfolio value, the engineering team must strictly adhere to the following final design parameters:

- 1. Optimized Local Architecture:** Replace heavy JVM dependencies with lightweight alternatives. Utilize standalone Debezium Server and Redpanda to drastically reduce Docker memory footprint. Ensure Apache Polaris is deployed as the central REST catalog to manage Iceberg concurrency safely.
- 2. Automated Lakehouse Maintenance:** Discard external batch orchestrators for table maintenance. Rely exclusively on the Apache Iceberg Sink V2 `addPostCommitTopology` within the Flink streaming job to automatically trigger `RewriteDataFiles`. This solves the Merge-on-Read small file problem natively, ensuring Trino query latency remains performant without adding infrastructure complexity.
- 3. Dynamic Dataset Strategy:** Abandon synthetic data generation. Utilize the Olist Brazilian E-Commerce dataset, replayed chronologically via a Python script to simulate real-world CDC streaming mutations, temporal delays, and null anomalies. This provides the necessary relational depth to challenge the AI models.
- 4. Empirical AI Benchmarking:** Pivot the project into a formal systems evaluation. Compare a Frontier LLM (Claude/GPT), an open-weight local SLM (Qwen2.5-Coder-7B), and a compiled deterministic Semantic Layer (Malloy) using a verified 50-question benchmark. Quantify the exact trade-offs between generative flexibility, execution accuracy, and query latency.
- 5. Deterministic AI Security:** Prove enterprise readiness by utilizing `sqlglot` to parse generated SQL strings into Abstract Syntax Trees. Programmatically intercepting and rejecting mutation commands demonstrates a profound understanding of AI safety boundaries that vastly exceeds rudimentary prompt engineering.

By aggressively managing scope to secure the Week 5 Minimum Viable Product, and subsequently executing the multi-engine experimental framework, the team will deliver an academically unassailable thesis. This capstone will not only prove mastery over complex, second-order distributed streaming mechanics but will also provide empirical data contributing to the broader industry dialogue on deploying LLMs safely over enterprise data lakes.

Works cited

1. Agentic Lakehouse Capstone Project.pdf
2. Debezium Server CDC - Mia-Platform Documentation, https://docs.mia-platform.eu/docs/products/fast_data/connectors/debezium_cdc
3. Postgres CDC failure and WAL filling up disk space? - Stack Overflow, <https://stackoverflow.com/questions/79464155/postgres-cdc-failure-and-wal-filling-up-disk-space>
4. Postgres Connector | Airbyte Documentation, <https://docs.airbyte.com/integrations/sources/postgres>
5. SEED: Enhancing Text-to-SQL Performance and Practical Usability Through

- Automatic Evidence Generation - arXiv, <https://arxiv.org/html/2506.07423v1>
6. Debezium without Kafka: Digging into the Debezium Server and Debezium Engine run times no one talks about : r/dataengineering - Reddit, https://www.reddit.com/r/dataengineering/comments/1kita4e/debezium_without_kafka_digging_into_the_debezium/
 7. Debezium Server — Guara Cloud Docs, <https://guaracloud.com/en/docs/catalog/debezium/>
 8. Building a Lightweight CDC Pipeline: Syncing MySQL to Meilisearch, <https://xylentis.com/blog/building-a-lightweight-cdc-pipeline-syncing-mysql-to-meilisearch-with-debezium-1>
 9. README.md · development · Iron Bank Containers / Opensource / debezium / server-2.1.4.final - Repo One, <https://repo1.dso.mil/dsop/opensource/debezium/server-2.1.4.final/-/blob/development/README.md>
 10. Debezium and Quarkus: Change Data Capture Patterns to Avoid Dual-Writes Problems, <https://www.infoq.com/articles/change-data-capture-debezium/>
 11. Pervasive Annotation Errors Break Text-to-SQL Benchmarks and Leaderboards - arXiv, <https://arxiv.org/html/2601.08778v3>
 12. The State of Streaming to Apache Iceberg in July 2026: Every Path, Its Latency, and What to Do When Seconds Are Not Fast Enough - DEV Community, <https://dev.to/alexmercedcoder/the-state-of-streaming-to-apache-iceberg-in-july-2026-every-path-its-latency-and-what-to-do-when-i6p>
 13. Flink TableMaintenance - Apache Iceberg™, <https://iceberg.apache.org/docs/nightly/flink-maintenance/>
 14. BEAVER: An Enterprise Benchmark for Text-to-SQL - arXiv, <https://arxiv.org/html/2409.02038v3>
 15. Apache Iceberg with Flink: Streaming Optimization Guide - LakeOps Blog, <https://lakeops.dev/blog/flink-iceberg-optimization>
 16. Flink Writes - Apache Iceberg™, <https://iceberg.apache.org/docs/nightly/flink-writes/>
 17. Flink Writes - Apache Iceberg™, <https://iceberg.apache.org/docs/1.9.1/flink-writes/>
 18. Merge-on-Read vs Copy-on-Write in Apache Iceberg - OLake, <https://olake.io/iceberg/mor-vs-cow/>
 19. BIRD Bench: The SQL Benchmark Built for Real Data - Beehive Advisors, <https://beehive-advisors.com/blog/bird-bench>
 20. Spec - Apache Iceberg™, <https://iceberg.apache.org/spec/>
 21. Why StarRocks is Better Than Trino for Building Open Lakehouse Architectures - Medium, <https://medium.com/@indomitability/why-starrocks-is-better-than-trino-for-building-open-lakehouse-architectures-d1a2c513871d>
 22. Delete Files vs Deletion Vectors in Apache Iceberg: How V3 Rewrote the Economics of Changing Data - DEV Community, <https://dev.to/alexmercedcoder/delete-files-vs-deletion-vectors-in-apache-iceberg-how-v3-rewrote-the-economics-of-changing-data-3i2b>
 23. Flink TableMaintenance - Apache Iceberg™,

- <https://iceberg.apache.org/docs/1.10.0/flink-maintenance/>
24. Self-Host a Text-to-SQL Data Analyst Agent on GPU Cloud (2026,
<https://www.spheron.network/blog/self-host-text-to-sql-data-analyst-agent-gpu-cloud/>
 25. MCP Tool Server (mcp_server.py) - GitHub,
<https://github.com/planetf1/mcp-server>
 26. FastMCP: Building MCP Servers in Python | MintMCP Blog,
<https://www.mintmcp.com/blog/fastmcp>
 27. Defining tools with MCP · Introduction to Model Context Protocol,
<https://academy.claude.com/courses/introduction-to-model-context-protocol/defining-tools-with-mcp>
 28. FastMCP 3.0: Build MCP Servers in Python, Fast | Apigene Blog,
<https://apigene.ai/blog/fastmcp>
 29. BAPPA: Benchmarking Agents, Plans, and Pipelines for Automated Text-to-SQL Generation, <https://arxiv.org/html/2511.04153v1>
 30. How Apache Iceberg's Optimizations Work and Why They Matter | by Luis Felipe | Medium,
https://medium.com/@luisfelipe_342/how-apache-icebergs-optimizations-work-and-why-they-matter-53d94a4d448c
 31. SynSQL: Synthesizing Relational Databases for Robust Evaluation of Text-to-SQL Systems, <https://arxiv.org/html/2604.27261v1>
 32. Spider 2.0-AIFunc: Extending Real-World Text-to-SQL to AI-Native SQL Workflows - arXiv, <https://arxiv.org/html/2607.06229v1>
 33. [2411.07763] Spider 2.0: Evaluating Language Models on Real-World Enterprise Text-to-SQL Workflows - arXiv, <https://arxiv.org/abs/2411.07763>
 34. Spider 2.0, <https://spider2-sql.github.io/>
 35. The Text-to-SQL Accuracy Cliff: Why Deterministic Compilers Beat LLM Guessing - Colrows, <https://colrows.com/blogs/text-to-sql-accuracy-cliff/>
 36. BIRD-INTERACT, <https://bird-interact.github.io/>
 37. [2510.05318] BIRD-INTERACT: Re-imagining Text-to-SQL Evaluation for Large Language Models via Lens of Dynamic Interactions - arXiv, <https://arxiv.org/abs/2510.05318>
 38. Brijesh403/olist-marketplace-analytics - GitHub,
<https://github.com/Brijesh403/olist-marketplace-analytics>
 39. Brazilian E-Commerce Public Dataset by Olist - Kaggle,
<https://www.kaggle.com/datasets/olistbr/brazilian-ecommerce>
 40. Customer Satisfaction on Olist Brazilian E-Commerce - RPubs,
<https://rpubs.com/Malakai-dev/1440272>
 41. Medallion Architecture Explained: A Hands-On Guide to Star Schema for BI Reporting,
<https://www.geopits.com/blog/medallion-architecture-explained-a-hands-on-guide-to-star-schema-for-bi-reporting>
 42. Olist E-Commerce Sales Analysis. — a portfolio | by Vania Elvina Afiati | Medium,
<https://medium.com/@vaniaelvinaa/olist-e-commerce-sales-analysis-7837d43f37da>

43. E-Commerce Data Analysis SQL Project: | by Ming S. Lu | Medium,
<https://medium.com/@ming.s.lu1617/e-commerce-data-analysis-sql-project-dc673b4348a2>
44. Judithokon/olist-ecommerce-sales-data-analysis-using-python - GitHub,
<https://github.com/Judithokon/olist-ecommerce-sales-data-analysis-using-python/blob/main/Olist%20Data%20Analysis.ipynb>
45. Retrieve Only Relevant Tables Whether Few or Many: Adaptive Table Retrieval Method, <https://arxiv.org/html/2605.18766v1>
46. How Far Do On-Prem Open LLMs Get on Text-to-SQL? A Cross-Family Size×Technique Frontier on BIRD - arXiv, <https://arxiv.org/html/2606.29733v1>
47. Training Natural Language to SQL Reasoning Model By Reinforcement Learning - arXiv, <https://arxiv.org/html/2504.08600v5>
48. CogniSQL-R1-Zero: Execution-Aligned Text-to-SQL - Emergent Mind,
<https://www.emergentmind.com/topics/cognisql-r1-zero>
49. jk200201/qwen2.5-coder-7b-bird-cot · Hugging Face,
<https://huggingface.co/jk200201/qwen2.5-coder-7b-bird-cot>
50. Qwen2.5-Coder Technical Report - arXiv, <https://arxiv.org/html/2409.12186v1>
51. What Is a Semantic Layer in Data? Meaning & Benefits | Airbyte,
<https://airbyte.com/blog/the-rise-of-the-semantic-layer-metrics-on-the-fly>
52. Lessons on building an AI data analyst - Pedro Nascimento,
<https://www.pedronasc.com/articles/lessons-building-ai-data-analyst>
53. Text-to-SQL vs the Semantic Layer: What the 2026 Benchmarks Show - Visivo,
<https://visivo.io/blog/text-to-sql-vs-semantic-layer>
54. Best Open-Source Semantic Layer Tools in 2026 | by Sergey Gromov - Level Up Coding,
<https://levelup.gitconnected.com/best-open-source-semantic-layer-tools-in-2026-f99565169ccc>
55. Malloy, <https://www.malloydata.dev/>
56. malloy/packages/malloy/CONTEXT.md at main - GitHub,
<https://github.com/malloydata/malloy/blob/main/packages/malloy/CONTEXT.md>
57. A sequel to SQL? An intro to Malloy - Hacker News,
<https://news.ycombinator.com/item?id=32738874>
58. Composability (ACR) - OrionBelt® Semantic Layer - ralforion,
<https://ralforion.com/orionbelt-semantic-layer/guide/composability/>
59. ReViSQL: Achieving Human-Level Text-to-SQL - arXiv,
<https://arxiv.org/html/2603.20004v1>
60. ReViSQL: Achieving Human-Level Text-to-SQL | alphaXiv,
<https://www.alphaxiv.org/abs/2603.20004>
61. GitHub - uiuc-kang-lab/text_to_sql_benchmarks: [VLDB2026] Pervasive Annotation Errors Break Text-to-SQL Benchmarks and Leaderboards,
https://github.com/uiuc-kang-lab/text_to_sql_benchmarks
62. Comparing Two Model Designs for Clinical Note Generation; Is an LLM a Useful Evaluator of Consistency? | Request PDF - ResearchGate,
https://www.researchgate.net/publication/382628269_Comparing_Two_Model_Designs_for_Clinical_Note_Generation_Is_an_LLM_a_Useful_Evaluator_of_Consistency

[cy](#)

63. PaVeRL-SQL: Text-to-SQL via Partial-Match Rewards and Verbal Reinforcement Learning, <https://arxiv.org/html/2509.07159v1>
64. Apache Iceberg with dbt: Optimization Guide - LakeOps Blog, <https://lakeops.dev/blog/dbt-iceberg-optimization>
65. [DE] Building a Lakehouse, and Leading the Team That Built It - Joshua Kim - Medium, <https://joshua-data.medium.com/building-a-lakehouse-and-leading-the-team-that-built-it-119e99a405b7>